UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingeniería de Software



CONWAY PARALLEL

Documentación de algoritmos

Simulación secuencial y paralela del Juego de la Vida

Programación Concurrente y Paralela

Grupo 01 | Periodo 2026-II

Integrantes

Bautista de la Cruz, Claudia	23200239
Bernal Hernández, Ryan	25200179
Ccahuana Quiñones, Valeria	23200248
Díaz Dionisio, Álvaro	11200142
Guzmán Chávez, Manuel	23200264
Hanampa Bellido, Iris	23200265
Munayco Sirlopu, Ronald	22200259
Rosales Trinidad, Jeanmarco	23200302

Lima, Perú — 2026

Algoritmo 1. Estado siguiente de una célula

NextCellState. Calcula el estado de una célula en la siguiente generación a partir de sus ocho vecinos. Aplica la regla B3/S23 y devuelve 1 si la célula estará viva o 0 si estará muerta. No modifica el tablero de entrada.

Algorithm 1: NextCellState

Data: *current*: `vector<uint8_t>`; *stride*, *row*, *col*: `size_t`
Result: *state*: `uint8_t`, with value 0 or 1

$i \leftarrow row \times stride + col$
$neighbors \leftarrow 0$
$neighbors \leftarrow neighbors + current[i - stride - 1]$
$neighbors \leftarrow neighbors + current[i - stride]$
$neighbors \leftarrow neighbors + current[i - stride + 1]$
$neighbors \leftarrow neighbors + current[i - 1]$
$neighbors \leftarrow neighbors + current[i + 1]$
$neighbors \leftarrow neighbors + current[i + stride - 1]$
$neighbors \leftarrow neighbors + current[i + stride]$
$neighbors \leftarrow neighbors + current[i + stride + 1]$
if $neighbors = 3$ ***or*** $(current[i] = 1$ ***and*** $neighbors = 2)$ **then**
  return 1
else
  return 0
end

Algoritmo 2. Evolución secuencial

SequentialStep. Calcula una generación completa con un único hilo. Recorre cada fila y columna, escribe los nuevos estados en el vector `next` y cuenta las células vivas. Al terminar, intercambia los buffers y actualiza la generación.

Algorithm 2: SequentialStep

Data: S: `GameState` (`struct`); $S.current$, $S.next$: `vector<uint8_t>`
Result: S: siguiente generación, población y contador actualizados

$nextAlive \leftarrow 0$
for $row \leftarrow 1$ **to** $S.height$ **do**
  for $col \leftarrow 1$ **to** $S.width$ **do**
    $i \leftarrow row \times S.stride + col$
    $state \leftarrow$ `NextCellState`$(S.current, S.stride, row, col)$
    $S.next[i] \leftarrow state$
    $nextAlive \leftarrow nextAlive + state$
  end
end
$S.aliveCount \leftarrow nextAlive$
`SwapBuffers`(S)
$S.generation \leftarrow S.generation + 1$

Algoritmo 3. Evolución paralela con OpenMP

OpenMPStep. Calcula una generación distribuyendo las filas entre varios hilos de OpenMP. Cada hilo lee el vector compartido `current`, escribe únicamente sus filas de `next` y acumula su población local. OpenMP combina los conteos mediante una reducción antes del intercambio de buffers.

Algorithm 3: OpenMPStep

Data: S: `GameState` (`struct`); $S.current$, $S.next$: `vector<uint8_t>`
Result: S: siguiente generación, población y contador actualizados

$nextAlive \leftarrow 0$
parallel region reduction$(+ : nextAlive)$ **do**
  omp for schedule(static) do
    for $row \leftarrow 1$ **to** $S.height$ **do**
      for $col \leftarrow 1$ **to** $S.width$ **do**
        $i \leftarrow row \times S.stride + col$
        $state \leftarrow$ `NextCellState`$(S.current, S.stride, row, col)$
        $S.next[i] \leftarrow state$
        $nextAlive \leftarrow nextAlive + state$
      end
    end
  end omp for
end parallel region
$S.aliveCount \leftarrow nextAlive$
`SwapBuffers`(S)
$S.generation \leftarrow S.generation + 1$